

```

1  (*
2                                     CS51 Lab 8
3                                     Functors
4                                     Part 2
5  *)
6
7  (* Objective:
8
9  This lab provides practice in the use of functors.
10 *)
11
12 (*=====
13 Functors - Part 2.
14
15 In this second part of the lab, you'll rewrite the `Stack` module from
16 the previous lab so that its element type is polymorphic, while adding
17 some functionality at the same time.
18
19 Occasionally, it's useful to be able to take a data structure and
20 "serialize" it (also called "marshalling"), that is, to transform its
21 internal computational representation into a format, like a string,
22 that can be saved in a file or transmitted digitally. It can later be
23 "deserialized" to recreate the internal representation. We'll add to
24 the stack module extremely basic support for serialization.
25
26 (You may notice that this provides an opportunity to break an
27 abstraction barrier: serializing an object, manipulating its string
28 representation, and then deserializing back to an object could
29 allow for some invariant to be violated. Even so, serialization has
30 its uses when the circumstances warrant.)
31
32 In order to serialize stacks from an abstract stack data type, we will
33 need to have access to a function that can serialize the *elements* of
34 the stack.
35
36 We can use a functor that takes as argument a module for the stack
37 elements (including element serialization functionality) and generates
38 a stack module that bundles together all of the stack functionality,
39 including serialization.
40
41 In order to do this, we'll first define a module interface, called
42 SERIALIZE, that captures the requirements for the elements of a
43 stack. It requires that the module expose both a data type and a
44 function (`serialize`) that converts values of that type into a string
45 representation. The SERIALIZE module type is an appropriate signature
46 for the argument of a functor that generates serializable stack
47 modules. *)
48
49 module type SERIALIZE =
50   sig
51     type t
52     val serialize : t -> string

```

```

53   end ;;
54
55   (* Now we'll define a STACK interface. Notice that unlike the
56   `INT_STACK` interface from the previous lab, we'll specify the
57   `element` type to be an abstract type as part of the signature, and
58   add additional functions for serialization, as well as a couple of
59   higher-order functions over stacks. *)
60
61   module type STACK =
62     sig
63       exception Empty
64       type element
65       type stack
66       val empty : stack
67       val push : element -> stack -> stack
68       val top : stack -> element
69       val pop : stack -> stack
70       val serialize : stack -> string
71       val map : (element -> element) -> stack -> stack
72       val filter : (element -> bool) -> stack -> stack
73       val fold_left : ('a -> element -> 'a) -> 'a -> stack -> 'a
74     end ;;
75
76   (*.....
77   Exercise 1A: Rewrite your stack implementation from the last lab to
78   use the `element` abstract data type for elements, and also complete
79   the other functions that the signature requires. We've provided a good
80   start below.
81
82   The serialize function should construct a string in the following
83   form:
84
85       "N0:N1:N2:...:N"
86
87   where N0 was the *first* element pushed onto the stack, N1 was the
88   *second*, and so on, with N being the *most recent* element pushed to
89   the stack, that is, the one to be popped next. (This would make
90   deserialization easier, since elements could be pushed as soon as they
91   were read.)
92
93   (Don't forget that you can implement other functions that are not
94   specified by the signature if you would like. The module signature
95   will act as an abstraction barrier to prevent the extra functions from
96   leaking out of the module.)
97   .....*)
98
99   module MakeStack (Element: SERIALIZE)
100     : (STACK with type element = Element.t) =
101     struct
102       exception Empty
103
104       type element = Element.t
105       type stack = element list
106

```

```

107   let empty : stack = []
108
109   let push (el : element) (s : stack) : stack =
110     failwith "push not implemented"
111
112   let pop_helper (s : stack) : (element * stack) =
113     failwith "pop_helper not implemented"
114
115   let top (s : stack) : element =
116     failwith "top not implemented"
117
118   let pop (s : stack) : stack =
119     failwith "pop not implemented"
120
121   let map (f : element -> element) (s : stack) : stack =
122     failwith "map not implemented"
123
124   let filter (f : element -> bool) (s : stack) : stack =
125     failwith "filter not implemented"
126
127   let fold_left (f : 'a -> element -> 'a) (init : 'a) (s : stack) : 'a =
128     failwith "fold_left not implemented"
129
130   let serialize (s : stack) : string =
131     failwith "serialize not implemented"
132 end ;;
133
134 (*.....
135 Exercise 1B: Now, make a module `IntStack` by applying the functor
136 that you just defined to an appropriate module for serializing integers.
137 .....*)
138
139 module IntStack = struct end ;;
140
141 (*.....
142 Exercise 1C: Make a module `IntStringStack` that creates a stack whose
143 elements are `int * string` pairs. Its serialize function should output
144 elements as strings of the form:
145
146     "(N, 'S')"
147
148 where N is the int, and S is the string. For instance, a stack with
149 two elements might be serialized as the string
150
151     "(1, 'pushed first'):(2, 'pushed second')".
152
153 For this oversimplified serialization function, you may assume that
154 the string will be made up of alphanumeric characters only.
155 .....*)
156
157 module IntStringStack = struct end ;;

```